Data Representation & Storing

IEEE standards for floating-point representation

what is a floating-point number?

- **Representation:** Floating-point numbers are a method of representing real numbers in a computer's memory. Unlike fixed-point representation, where the decimal point is fixed, in floating-point representation, the decimal point can "float" to represent numbers of varying magnitudes.

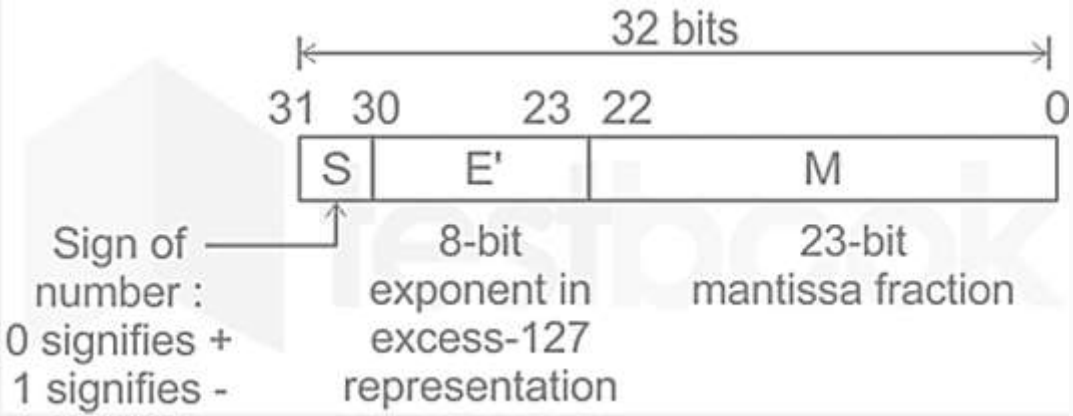
- **Structure:** A floating-point number typically consists of three parts:

    1. Sign bit: Determines whether the number **is positive or negative.**
    2. Exponent: Specifies the scale or magnitude of the number.
    3. Significand (or mantissa): The significant digits of the number **(represent the fraction part of the number)**

1. Single precision representation (32-bit word):

The single-precision floating-point representation is a 32-bit representation. It can be divided into three fields:

- **Field 1:** Sign → 1-bit (represents the sign of the number i.e. positive or negative.
- **Field 2:** Exponent→ 8-bit (signed exponent in excess-127 representation).
- **Field 3:** Mantissa fraction 23-bit The standard representation is shown below.



Example

The floating-point number (+3.5) can be represented in binary using IEEE 754 single-precision (32-bit) format:

Steps:

1. Set the **sign bit to 0**, since the number is **positive**.
2. Convert the number into binary format.
   - $(3.5)_{10} = (11.1)_2$
3. Normalize the number by moving the decimal point to the **left** so that there is only one non-zero digit to the left of the decimal point.
   - $(3.5)_{10} = (11.1)_2 = 1.11 \times 2^1$

4. Here, exponent = 1
   - ➢ **Excess 127 form of exponent = 127+1 = $(128)_{10}$ = $(10000000)_2$**
5. Mantissa = 11 , Convert the mantissa into binary format.
   - ➢ Mantissa = **11000000000000000000000** (with 23 trailing zeros)

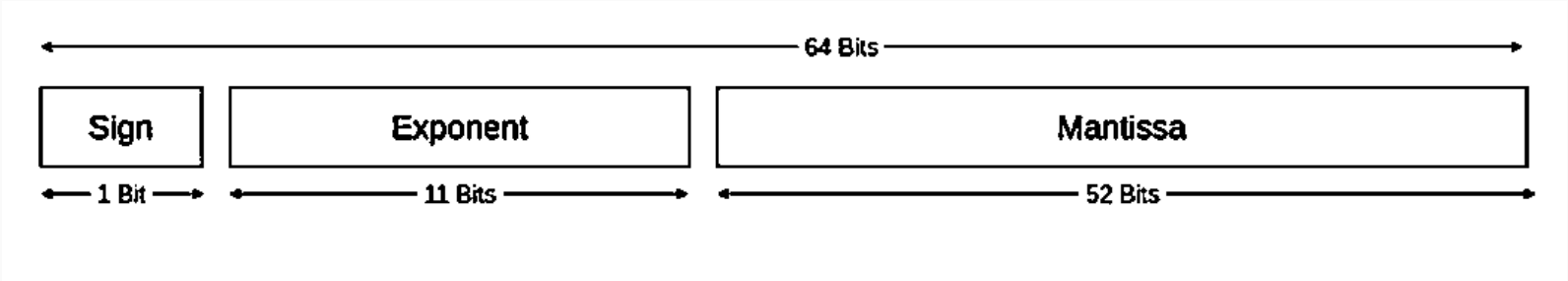
0 10000000 11000000000000000000000

- Try to solve:
  - $(-1.75)_{10}$
  - $(+111.625\ )_{10}$

2. Double precision representation (64-bit word):

For double-precision floating-point representation, a 64-bit format is typically used. It can also be divided into three fields:

- **Field 1:** Sign → 1-bit (represents the sign of the number i.e., positive or negative).
- **Field 2:** Exponent → 11-bit (signed exponent in excess-1023 representation).
- **Field 3:** Mantissa fraction → 52-bit .



Example

The floating-point number (+3.5) can be represented in binary using IEEE 754 double-precision (64-bit) format:

Steps:

1. Set the **sign bit to 0**, since the number is **positive**.
2. Convert the number into binary format.
   - $(3.5)_{10} = (11.1)_2$
3. Normalize the number by moving the decimal point to the **left** so that there is only one non-zero digit to the left of the decimal point.
   - $(3.5)_{10} = (11.1)_2 = 1.11 \times 2^1$

4. Here, exponent = 1
   - **Excess 1023 form of exponent = 1023+1 = $(1024)_{10}$ = $(10000000000)_2$**
5. Mantissa = 11 , Convert the mantissa into binary format.
   - **Mantissa =**

     11000000000000000000000000000000000000000000000000

     (pad the remaining bits with zeros)

0 10000000000 11000000000000000000000000000000000000000000000000

- Try to solve:
  - $(+4.5)_{10}$
  - $(-161.875)_{10}$

Thanks!